

Activity 1

Building a Scalable, Data-Driven Selenium Automation Framework

Solution Architecture, Design Decisions and Java/TestNG/Maven Implementation

Scope: Web UI automation only

1. Requirements Summary

Requirements were gathered through Socratic questioning before implementation. The following are the selected requirements.

- Chrome + Edge + Firefox + Safari; multiple environments; parallel execution.
- Browser/environment selection through TestNG XML + Maven parameters + configuration files.
- CSV test data with a reusable CSV utility feeding TestNG DataProvider.
- ThreadLocal for parallel isolation.
- Selenium Manager locally + RemoteWebDriver for distributed execution.
- Separate environment properties with Maven/system/environment-variable overrides.
- Page Objects + reusable components + BasePage + Page Factory.
- Implicit + explicit + fluent waits, governed by framework guidelines.
- TestNG + Extent Reports + Allure; screenshots on failure and configurable checkpoints.
- SLF4J + Log4j2 logging.
- TestNG groups + XML suites + Maven profiles.
- CSV loaded once and exposed as immutable test data.
- Configurable retry with selective retry for transient failures.
- Jenkins + GitHub Actions CI/CD.
- Encrypted configuration for sensitive values.
- Local + Selenium Grid + cloud-ready driver architecture.
- Maven BOM + dependencyManagement + version properties.
- ThreadLocal runtime context + immutable shared configuration.
- Configurable per-method/per-class WebDriver lifecycle.
- Page-specific and reusable component locators.
- BasePage + dedicated utility classes for common Selenium actions.
- Custom framework exceptions + fail-fast framework handling + centralized reporting.
- TestNG groups + custom annotations + external test metadata.
- Independent tests by default; explicit dependencies only where required.
- Common CSV + environment-specific configuration.
- Configurable parallel strategy.
- Reports include results, screenshots, logs, test data, browser and environment metadata.
- AI/self-healing locator recovery as a controlled fallback.
- Independent tests continue after failures.
- No API automation.

2. Solution Architecture

The framework uses layered architecture so test cases remain focused on business behavior while browser, configuration, data, synchronization, reporting and infrastructure concerns stay reusable.

```
TestNG Tests
|
Page Objects + Reusable Components
|
BasePage + Utilities + Wait/Locator Layer
|
DriverFactory + ThreadLocal Driver
|
Local Selenium Manager / RemoteWebDriver / Grid
|
Configuration + CSV Data + Reporting + Logging
```

Execution flow: Maven/TestNG parameters select browser and environment → configuration is resolved → a thread-local driver is created → immutable CSV data is supplied → Page Objects execute business actions → listeners capture evidence → reports are published.

3. Major Design Decisions

- **ThreadLocal driver:** isolates browser sessions during parallel execution.
- **CSV + DataProvider:** separates test data from test logic and supports parameterized execution.
- **Page Objects + components:** isolates UI changes and reduces duplication.
- **Environment configuration:** supports QA/UAT/Production without changing test code.
- **Maven profiles:** supports smoke, regression and cross-browser execution.
- **Dual reporting:** Extent and Allure provide complementary reporting capabilities.
- **Selective retry:** avoids masking genuine functional failures.
- **Self-healing:** controlled fallback only; stable locators remain the primary strategy.
- **Failure handling:** independent tests continue so regression execution provides maximum coverage.

4. Recommended Maven Project Structure

```
selenium-data-driven-framework/  
  pom.xml  
  README.md  
  .github/workflows/automation.yml  
  src/main/java/com/framework/  
    config/          ConfigManager.java  
    driver/          DriverManager.java / DriverFactory.java  
    pages/           BasePage.java / LoginPage.java  
    components/  
    data/            CsvReader.java / TestDataProvider.java  
    utils/           WaitUtils.java / ScreenshotUtils.java  
    reporting/  
    exceptions/  
  src/main/resources/  
    config/qa.properties  
    config/uat.properties  
    config/prod.properties  
    log4j2.xml  
  src/test/java/com/tests/  
    base/BaseTest.java  
    tests/LoginTest.java  
  src/test/resources/  
    testdata/login.csv  
    suites/smoke.xml / regression.xml
```

5. Core Implementation

Maven dependency management

```
<properties>  
  <java.version>17</java.version>  
  <selenium.version>4.x</selenium.version>  
  <testng.version>7.x</testng.version>  
  <slf4j.version>2.x</slf4j.version>  
  <log4j.version>2.x</log4j.version>  
</properties>  
  
<dependencyManagement>  
  <!-- Use approved BOMs where available. -->  
</dependencyManagement>  
  
<dependencies>  
  selenium-java  
  testng  
  slf4j-api  
  log4j-slf4j2-impl  
  reporting dependencies  
</dependencies>
```

Thread-safe WebDriver

```
public final class DriverManager {  
    private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();  
  
    public static void setDriver(WebDriver driver) { DRIVER.set(driver); }  
  
    public static WebDriver getDriver() {  
        WebDriver d = DRIVER.get();  
        if (d == null) throw new FrameworkException("Driver not initialized");  
        return d;  
    }  
  
    public static void quitDriver() {  
        WebDriver d = DRIVER.get();  
        if (d != null) d.quit();  
        DRIVER.remove();  
    }  
}
```

Browser factory

```
public final class DriverFactory {  
    public static WebDriver create(String browser, String mode) {  
        if ("remote".equalsIgnoreCase(mode)) return createRemote(browser);  
  
        return switch (browser.toLowerCase()) {  
            case "chrome" -> new ChromeDriver();  
            case "edge" -> new EdgeDriver();  
            case "firefox" -> new FirefoxDriver();  
            case "safari" -> new SafariDriver();  
            default -> throw new FrameworkException("Unsupported browser: " + browser);  
        };  
    }  
  
    private static WebDriver createRemote(String browser) {  
        // Build browser options/capabilities and use RemoteWebDriver.  
        throw new UnsupportedOperationException("Configure Grid endpoint");  
    }  
}
```

}

Safari requires a supported macOS execution target; on Windows it should be routed to a compatible Grid/cloud node.

6. Page Object Model + Page Factory

```
public abstract class BasePage {
    protected final WebDriver driver;
    protected final WebDriverWait wait;

    protected BasePage() {
        driver = DriverManager.getDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        PageFactory.initElements(driver, this);
    }

    protected void click(WebElement element) {
        wait.until(ExpectedConditions.elementToBeClickable(element)).click();
    }

    protected void type(WebElement element, String value) {
        wait.until(ExpectedConditions.visibilityOf(element));
        element.clear();
        element.sendKeys(value);
    }
}

public class LoginPage extends BasePage {
    @FindBy(id="username") private WebElement username;
    @FindBy(id="password") private WebElement password;
    @FindBy(css="[data-testid='login']") private WebElement loginButton;

    public LoginPage enterUsername(String value) {
        type(username, value); return this;
    }

    public LoginPage enterPassword(String value) {
        type(password, value); return this;
    }

    public DashboardPage login() {
        click(loginButton);
        return new DashboardPage();
    }
}
```

Stable attributes such as dedicated test IDs should be preferred. The selected self-healing layer should only act as a controlled fallback and should log every recovery event.

7. Data-Driven CSV Testing

```
username,password,expected
user1,password1,success
user2,password2,success
invalid,wrong,validation_error

public record LoginData(String username, String password, String expected) {}

@DataProvider(name="loginData", parallel=true)
public Object[][] loginData() {
    return CsvCache.getLoginData().stream()
        .map(d -> new Object[]{d})
        .toArray(Object[][]::new);
}

@Test(dataProvider="loginData", groups={"smoke","regression"})
public void loginTest(LoginData data) {
    new LoginPage()
        .enterUsername(data.username())
        .enterPassword(data.password())
        .login();
}
```

The CSV should be loaded once into immutable records. This improves performance and avoids shared mutable state during parallel execution.

8. Configuration and Environment Handling

```
# qa.properties
base.url=https://qa.example.com
driver.mode=local
explicit.wait=15

# uat.properties
base.url=https://uat.example.com

# prod.properties
base.url=https://prod.example.com

public final class ConfigManager {
    private static final Properties PROPS = new Properties();

    public static void load(String environment) {
        String file = "/config/" + environment + ".properties";
        try (InputStream in = ConfigManager.class.getResourceAsStream(file)) {
            if (in == null) throw new FrameworkException("Missing config: " + file);
            PROPS.load(in);
        } catch (IOException e) {
            throw new FrameworkException("Config load failed", e);
        }
    }

    public static String get(String key) {
        String v = System.getProperty(key);
        if (v == null || v.isBlank()) v = System.getenv(key);
        return v != null ? v : PROPS.getProperty(key);
    }
}
```

9. TestNG Lifecycle, Groups and Parallel Execution

```
@BeforeMethod(alwaysRun=true)
@Parameters({"browser", "environment"})
public void setUp(String browser, String environment) {
    ConfigManager.load(environment);
    WebDriver driver = DriverFactory.create(
        browser, ConfigManager.get("driver.mode"));
    DriverManager.setDriver(driver);
    driver.manage().window().maximize();
    driver.get(ConfigManager.get("base.url"));
}

@AfterMethod(alwaysRun=true)
public void tearDown() {
    DriverManager.quitDriver();
}
```

The default lifecycle shown is per-method for maximum isolation. The selected configurable lifecycle can additionally support per-class execution where performance data justifies browser reuse.

10. Wait Strategy

Implicit, explicit and fluent waits are supported because that was selected. However, the framework should avoid arbitrary sleeps, keep implicit waits minimal/zero, and use explicit condition-based waits as the normal mechanism.

```
public static WebElement fluent(WebDriver driver, By locator) {
    Wait<WebDriver> wait = new FluentWait<>(driver)
        .withTimeout(Duration.ofSeconds(20))
        .pollingEvery(Duration.ofMillis(500))
        .ignoring(NoSuchElementException.class);
    return wait.until(d -> d.findElement(locator));
}
```

11. Reporting, Logging, Screenshots and Retry

Reports should contain result status, screenshots, logs, test-data identifier, browser, environment and execution metadata. Failure screenshots are automatic; successful checkpoints are configurable.

```
public class TransientRetryAnalyzer implements IRetryAnalyzer {
    private int attempts = 0;
    private final int maxRetries = 1;

    @Override
    public boolean retry(ITestResult result) {
        if (attempts >= maxRetries) return false;

        Throwable t = result.getThrowable();
        boolean transientFailure =
            t instanceof TimeoutException ||
            t instanceof WebDriverException;

        if (transientFailure) {
            attempts++;
            return true;
        }
        return false;
    }
}
```

Retry must remain visible in reports and must not be used to hide repeatable assertion failures.

Logging uses SLF4J as the abstraction and Log4j2 as the implementation. Framework and test events should use appropriate INFO/DEBUG/WARN/ERROR levels.

12. CI/CD

```
mvn clean test -Psmoke -Dbrowser=chrome -Denvironment=qa
mvn clean test -Pregression -Dbrowser=firefox -Denvironment=uat
```

Jenkins and GitHub Actions can invoke the same Maven commands. Environment variables and CI/CD secrets should supply runtime values without modifying source code.

13. Security

Encrypted configuration is used for sensitive values. The encryption/decryption key must never be stored beside the encrypted file; it should be injected securely by the execution environment.

14. Self-Healing Locator Strategy

```
public interface LocatorResolver {
    WebElement resolve(WebDriver driver, By primary, List<By> fallbacks);
}

public final class HealingLocatorResolver
    implements LocatorResolver {

    public WebElement resolve(
        WebDriver driver, By primary, List<By> fallbacks) {
        try {
            return driver.findElement(primary);
        } catch (NoSuchElementException e) {
            for (By fallback : fallbacks) {
                try {
                    WebElement element = driver.findElement(fallback);
                    log.warn("Locator recovered: {} -> {}", primary, fallback);
                    return element;
                } catch (NoSuchElementException ignored) {}
            }
            throw e;
        }
    }
}
```

A production AI/self-healing provider should be replaceable and observable. Stable locators remain the primary strategy; healing should never silently hide application defects.

15. Alternatives and Trade-offs

- **Page Factory:** retained because it was explicitly selected; plain By-based Page Objects are often simpler in newer designs.
- **Wait combination:** supported, but explicit waits should be the normal synchronization mechanism.
- **Extent + Allure:** powerful but adds maintenance; a lean framework could use one reporting tool.
- **Self-healing:** reduces locator maintenance but can hide defects; every recovery must be logged/reported.
- **Encrypted files:** protect data at rest, but key management remains critical.
- **Safari:** requires a compatible macOS target; use Grid/cloud for Windows-based development.
- **Parallel execution:** improves throughput but requires strict thread isolation and immutable data.

16. Best Practices Checklist

- Prefer stable test attributes and semantic locators.
- Keep tests independent by default.
- Avoid Thread.sleep; synchronize on meaningful UI conditions.
- Keep WebDriver and runtime state thread-local.
- Never commit plain-text credentials.
- Use immutable shared test data.
- Keep Page Objects focused on business actions.
- Centralize common Selenium interactions.
- Keep retry limited and observable.
- Attach screenshots/logs to failures.
- Publish CI/CD reports as artifacts.
- Use Maven profiles and external parameters for execution variation.
- Treat self-healing as controlled fallback, not a substitute for locator quality.

17. Final Execution Flow

```
Maven / CI
-> TestNG Suite + Groups
-> Environment + Browser Resolution
-> ThreadLocal WebDriver
-> Immutable CSV DataProvider
-> Test -> Page Object -> Component -> BasePage
-> Wait / Locator / Healing Layer
-> Logs + Screenshots + Retry + Metadata
-> Extent + Allure
-> Jenkins / GitHub Actions Artifacts
```

Conclusion: The proposed architecture separates test intent from automation infrastructure, making it suitable for scalable data-driven UI automation, cross-browser execution, parallel runs and CI/CD.

Submission: Activity 1 – Scalable, Data-Driven Selenium Automation Framework